

Virtual Memory – Demand Paging – Process creation – Page Replacement – Allocation of frames – Thrashing. File Concept: Access Methods – Directory Structure – File System Mounting – File Sharing – Protection

1. What is virtual memory?(APR '11)(APR '12)

Virtual memory is a technique that allows the execution of processes that may not be completely in memory. It is the separation of user logical memory from physical memory. This separation provides an extremely large virtual memory, when only a smaller physical memory is available.

2. What is Demand paging? (NOV '18)

Virtual memory is commonly implemented by demand paging. In demand paging, the pager brings only those necessary pages into memory instead of swapping in a whole process. Thus it avoids reading into memory pages that will not be used anyway, decreasing the swap time and the amount of physical memory needed.

3. Define lazy swapper.

Rather than swapping the entire process into main memory, a lazy swapper is used. A lazy swapper never swaps a page into memory unless that page will be needed.

4. Compare Swapper with Lazy Swapper?(NOV '13)

Rather than swapping the entire process into main memory, a lazy swapper is used. A lazy swapper never swaps a page into memory unless that page will be needed.

5. What is a pure demand paging?

When starting execution of a process with no pages in memory, the operating system sets the instruction pointer to the first instruction of the process, which is on a non-memory resident page, the process immediately faults for the page. After this page is brought into memory, the process continues to execute, faulting as necessary until every page that it needs is in memory. At that point, it can execute with no more faults. This schema is pure demand paging.

6. Define secondary memory.

This memory holds those pages that are not present in main memory. The secondary memory is usually a high speed disk. It is known as the swap device, and the section of the disk used for this purpose is known as swap space.

7. Consider a logical address space of eight pages of 1024 words each, mapped onto a physical memory of 32 frames. A. How many bits are there in the logical address? B. How many bits are there in the physical address?

Answer:

A. Logical address: 13 bits

B. Physical address: 15 bits

8. What is the basic approach of page replacement? If no frame is free is available, find one that is not currently being used and free it.

A frame can be freed by writing its contents to swap space, and changing the page table to indicate that the page is no longer in memory. Now the freed frame can be used to hold the page for which the process faulted.

9. What is the various page replacement algorithms used for page replacement?

- FIFO page replacement
- Optimal page replacement
- LRU page replacement
- LRU approximation page replacement
- Counting based page replacement
- Page buffering algorithm.

10. What are the major problems to implement demand paging?

The two major problems to implement demand paging is developing

- a. Frame allocation algorithm
- b. Page replacement algorithm

11. What is a reference string?

An algorithm is evaluated by running it on a particular string of memory references and Computing the number of page faults. The string of memory reference is called a reference string.

12. What is thrashing? (APR 11, NOV '15)

Thrashing occurs when a computer's virtual memory subsystem is in a constant state of paging, rapidly exchanging data in memory for data on disk, to the exclusion of most application-level processing. This causes the performance of the computer to degrade or collapse. The situation may continue indefinitely until the underlying cause is addressed.

13. Write any two causes of Thrashing?

In virtual memory systems, thrashing may be caused by programs or workloads that present insufficient locality of reference: if the working set of a program or a workload cannot be effectively held within physical memory, then constant data swapping, *i.e.*, thrashing, may occur. The term was first used during the tape operating system days to describe the sound the tapes made when data was being rapidly written to and read from them. Many older low-end computers have insufficient RAM (memory) for modern usage patterns and increasing the amount of memory can often cause the computer to run noticeably faster. This speed increase is due to the reduced amount of paging necessary.

14. State the functions of working set strategy model (NOV'14)

To prevent thrashing, we must provide a process with as many frames as it needs. To find how many frames a process needs for that a technique called working set strategy used. The working-set strategy starts by looking at how frames a process is actually using. This approach defines the locality of process execution. The locality model states that, as a process executes, it moves from locality to locality. A locality is a set of pages that are actively used together. Allocate enough frames to a process to accommodate its current locality. It will fault for the pages in its locality until all these pages are in memory; then, it will not fault again until it changes localities.

15. Enumerate Search Path? (NOV '13)

When you enter a command, the operating system has to search for that program. The path defines the directories which the system will search. The current directory is NOT automatically included in the search path.

To add the directory /usr/local/bin to the existing path, use the command:
PATH=\$PATH:/usr/local/bin

16. What is a file?

A file is a named collection of related information that is recorded on secondary storage. A file contains either programs or data. A file has certain "structure" based on its type.

- File attributes: Name, identifier, type, size, location, protection, time, date
- File operations: creation, reading, writing, repositioning, deleting, truncating, appending, renaming
- File types: executable, object, library, source code etc.

17. What is Access method?

An access method defines the technique that is used to store and retrieve data. Access methods have their own data set structures to organize data, system-provided programs (or **macros**) to define data sets, and utility programs to process data sets.

18. What are the possible methods to be used to access file?

There are two possibilities: sequential access and random access.

Sequential access: Each access to a given file starts where the previous access to that file finished (the first access to the file starts at the beginning of the file). Sequential access is the most common and gives the highest performance. For some devices (e.g. magnetic or paper tape) access must be sequential.

Random access: The bytes are accessed in any order. Thus each access must specify which bytes are desired. This is done either by having each read/write specify the starting location or by defining another system call (often named seek) that specifies the starting location for the next read/write.

19. List the various File Attributes? (APR '12)(NOV '12)

- **Name** – only information kept in human-readable form
- **Identifier** – unique tag (number) identifies file within file system
- **Type** – needed for systems that support different types
- **Location** – pointer to file location on device
- **Size** – current file size
- **Protection** – controls who can do reading, writing, executing
- **Time, date, and user identification** – data for protection, security, and usage monitoring

20. What are the various file operations?

- **Create:** The effect of create is essential if a system is to add files. However, it need not be a separate system call. (For example, it can be merged with open).
- **Delete:** Essential, if a system is to delete files.
 - **Open:** Not essential. An optimization in which a process translates a file name to the corresponding disk locations only once per execution rather than once per access. We shall see that for the Unix inode-based file systems, this translation can be quite expensive.
 - **Close:** Not essential. Frees resources without waiting for the process to terminate.
 - **Read:** Essential Must specify filename, file location, number of bytes, and a buffer into which the data is to be placed. Several of these parameters can be set by other system calls and in many operating systems they are.
 - **Write:** Essential, if updates are to be supported. See read for parameters.
 - **Seek:** Not essential (could be in read/write). Specify the offset of the next (read or write) access to this file.
 - **Get attributes:** Essential if attributes are to be used.
 - **Set attributes:** Essential if attributes are to be user settable.
 - **Rename:** Copy and delete is not an acceptable substitute for big files. Moreover, copy-delete is not atomic.

21. Write some file type and its extension?

File Type	Usual extension	Function
executable	Exe,com,bin or none	Read to run machine language program
object	Obj,O	Compiled ,machine language, not linked
Source code	c,cc,java,pas,asm,a	Source code in various language
batch	Bat,sh	Commands to the command interpreter
text	txt,doc	Textual data,documents
Word processor	Wp,tex,rrf,doc	Various word processor formats
library	Lib,a,mpeg,mov,rm	Libraries of routines for programmers.

22.What are the information associated with an open file?

Several pieces of information are associated with an open file which may be:

- File pointer
- File open count
- Disk location of the file
- Access rights

23. What are the different accessing methods of a file? (APR'15)

The different types of accessing a file are:

- Sequential access: Information in the file is accessed sequentially
- Direct access: Information in the file can be accessed without any particular order.
- Other access methods: Creating index for the file, indexed sequential access method (ISAM) etc.

24.What is Access control?

The most common approach to the protection problem is to make access dependent on the identity of the user. Different users may need different types of access to a file or directory. The most general scheme to implement dependent access is to associate with each file and directory specifying user names and the types of access allowed for each user. When a user requests access to a particular file, the operating system checks the access list associated with that file. If that user is listed for the requested access, the access is allowed. Otherwise, a protection violation occurs, and the user job is denied access to the file.

25.What are the advantages and disadvantages of access control? (NOV'14)

The access control approach has the advantage of enabling complex access methodologies. The main problem with access lists is their length. If we want to allow everyone to read a file, we must list all users with read access. This technique has two undesirable consequences:

1. Constructing such a list may be a tedious and unrewarding task, especially if we do not know in advance the list of users in the system.
2. The directory entry, previously of fixed size, now must be of variable size, resulting in more complicated space management.

26. What is Directory?

The device directory or simply known as directory records information-such as name, location, size, and type for all files on that particular partition. The directory can be viewed as a symbol table that translates file names into their directory entries. OR Directory is collection of files. **Directories** maintain information about files: For a large number of files, may want a directory structure is directories under directories.

27. What are the operations that can be performed on a directory?

The operations that can be performed on a directory are

- Search for a file

- Create a file
- Delete a file
- Rename a file
- List directory
- Traverse the file system

28. What are the most common schemes for defining the logical structure of a directory?

The most common schemes for defining the logical structure of a directory

- Single-Level Directory
- Two-level Directory
- Tree-Structured Directories
- Acyclic-Graph Directories
- General Graph Directory

29. Define UFD and MFD.

In the two-level directory structure, each user has her own user file directory (UFD). Each UFD has a similar structure, but lists only the files of a single user. When a job starts the system's master file directory (MFD) is searched. The MFD is indexed by the user name or account number, and each entry points to the UFD for that user.

30. What is a path name?

A pathname is the path from the root through all subdirectories to a specified file. In a two-level directory structure a user name and a file name define a path name.

31. What is System file checker in Windows?(NOV'11)

System File Checker (sfc) is a utility in Windows that allows users to scan for corruptions in Windows system files and restore corrupted files. Scans and verifies the versions of all protected system files after you restart your computer.

32. Give any two criteria to chose a file organization? (APR '12)

1. Fast access to single record or collection of related records.
2. Easy record adding, updating, removal without disrupting.
3. Storage efficiency.

33. How information is maintained in is Directory?

Information maintained in a directory:

Name	The user visible name.
Type	The file is a directory, a program image, a user file, a link, etc.
Location	Device and location on the device where the file header is located.
Size	Number of bytes/words/blocks in the file.
Position	Current next-read/next-write pointers.
Protection	Access control on read/write/execute/delete
Usage	time of creation/access, etc.

34. What is Directory Structure?

In an Partitions (or Volumes) can be viewed as the abstraction of virtual disks. Disks can be partitioned into separate areas such that each partition is treated as a separate storage device. The other way a partition may be defined to be more than one disk device than one. Partitions can store multiple operating systems such that a system can boot more than one OS. Each partition contains information about files in a device directory (or) a VTOC (Volume Table of Contents) is a device directory. Each directory records file attribute information.

35. What are the operations that can be performed on a directory?

Search for a file – need to find a particular entry or be able to find file names based on a pattern match.

Create a file -and add its entry to the directory.

Delete a file – and remove remove it from the directory.

List a directory –list both the files in the directory and the directory contents for each file.

Rename a file –renaming may imply changing the position of the file entry in the directory structure.

Traverse the file system –the directory needs a logical structure such that every directory and every file within each directory can be accessing efficiently.

36. Mention the Directory design goals?

- Efficiency to locating a file quickly.
- Naming is convenient to users.
 - ◆ Two users can have same name for different files.
 - ◆ The same file can have several different names.
- Grouping in a logical grouping of files by properties, (e.g., all Java programs, all games, ...)

37. What is File Lock? (NOV '11) (APR '11)

File locking is a mechanism that restricts access to a computer file by allowing only one user or process access at any specific time. Systems implement locking to prevent the classic interceding update scenario (race condition). It may also refer to additional security applied by a computer user either by using Windows security, NTFS permissions or by installing a third party file locking software.

38. What is File Control Block? (APR '14)

A File Control Block (FCB) is a file system structure in which the state of an open file is maintained. An FCB is managed by the operating system, but it resides in the memory of the program that uses the file, not in operating system memory. This allows a process to have as many files open at one time as it wants to, provided it can spare enough memory for an FCB per file.

39. Define Single-Level Directory?

A single-level directory with file entries for all users contained in the same directory.

Advantages:

Easy to support and understand.

Disadvantages:

Requires unique file names unique file names unique file names {the naming problem}.

No natural system for keeping track of file names {the grouping problem}

40. Define Two Level Directory?

Two Level Directory is a separate directory for each user. The system's Master File Directory (MFD) Master File Directory (MFD) Master File Directory (MFD) has pointers to individual User File Directories (UFD's).

File names default to localized UFD for all operations.

Advantages:

- Solves the name-collision problem.
- Isolates users from one another a form of protection.
- Efficient searching.

Disadvantages:

- Restricts user cooperation.
- No logical grouping capability (other than by user).

41. Define Tree-Structured Directories?

This generalization to a directory tree structure of arbitrary height allows users to create their own subdirectories and organize their files accordingly.

Advantages:

- Efficient searching
- Grouping Capability
- Each user has a current directory(working directory)

cd /spell/mail/prog

Type list

42. Define Acyclic-Graph Directories?

A tree structure prohibits the sharing of files or directories. Acyclic graphs allow directories to have shared subdirectories and files.

43. What is File System Mounting? (NOV '18)

A file system must be mounted before it can be available to processes on the system. The mount procedure the OS is given the device name and the location within the file structure name location within the file structure location within the file structure at which to attach the the file system. {the mount point}.A mount point is typically an empty directory where the mounted file system will be attached.The OS verifies that device has valid file system by asking device driver to read the device directory and verify that directory has the proper format.

44. What is File Authentication?(NOV '11)(ARPIL '11)

An important aspect of security on a computer system is the granting or denying of permissions (sometimes called access rights). Permission is the ability to perform a specific operation such as to gain access to data or to execute code. Permissions can be granted at the level of directories, subdirectories, files or applications, or specific data within files or functions within **applications**.

45. Define File Sharing?

- Sharing of files on multi-user systems is desirable.
- Sharing may be done through a protection.
- On distributed systems, files may be shared across a network .
- Network File System (NFS) is a common distributed file sharing method.

46. How will you protect the file system?

Protecting the File system the File owner/creator should be able to control: what can be done and by whom

The following access methods are to protecting file

1. Read
2. Write
3. Execute
4. Append
5. Delete
6. List

47. What is the principle of optimality? (NOV '15)

Optimality is the best solution that can be achieved. In page replacement algorithm, the best replacement technique can be achieved by using optimal page replacement algorithm. It is simply replaces the page that will not be used for the longest period of time.

11 MARKS

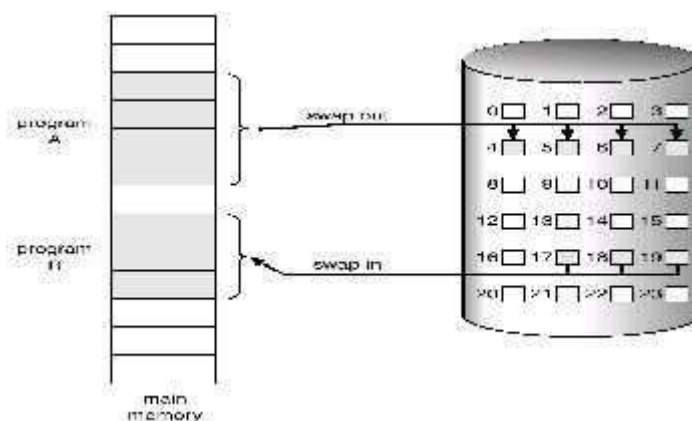
1. Explain demand paging in detail? (11)

Demand paging:

As there is much less physical memory than virtual memory the operating system must be careful that it does not use the physical memory inefficiently. One way to save physical memory is to only load virtual pages that are currently being used by the executing program. For example, a database program may be run to query a database. In this case not the entire database needs to be loaded into memory, just those data records that are being examined. Also, if the database query is a search query then it does not make sense to load the code from the database program that deals with adding new records. This technique of only loading virtual pages into memory as they are accessed is known as demand paging.

Transfer of a paged memory to contiguous disk space

- It is similar in paging system with swapping.
- Processes reside on secondary memory, when we want to execute a process, we swap it into memory.
- Rather than swapping the entire process into memory, we can use a **lazy swapper**.
- A lazy swapper never swaps a page into memory unless that page will be needed. Swapper that deals with pages is a pager.
- When a process is to be swapped in, instead of swapping a whole process the page brings in only those necessary pages into memory.



- Thus it avoids reading into memory pages that will not be used anyway, decreasing the swap time and the amount of physical memory needed.
- With this scheme we need some hardware support to distinguish between those pages that are in memory and those pages that are in disk.
- The valid, invalid bit scheme can be used for this purpose.

- When the bit is set it valid it indicates that the associated is both legal and in memory.
- If the bit set to invalid is not valid or is valid but not in memory.

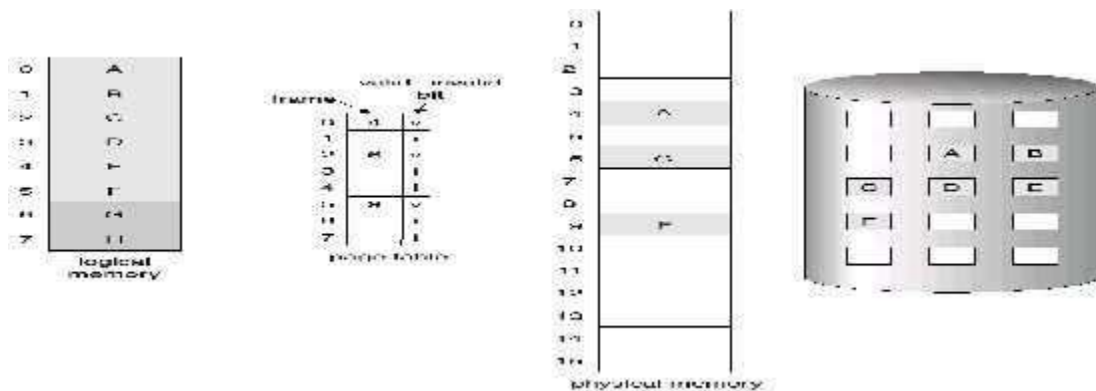
Page table when some pages are not in main memory:

- With each page table entry a valid-invalid bit is associated (v $\Rightarrow$ in-memory, i $\Rightarrow$ not-in-memory)
- Initially valid-invalid bit is set to i on all entries.
- During address translation, if valid-invalid bit in page table entry is **I** $\Rightarrow$ page fault

Example of a page table snapshot:

If there is a reference to a page, first reference to that page will trap to operating system:page fault

1. Operating system looks at another table to decide:
 - Invalid reference $\Rightarrow$ abort
 - Just not in memory
2. Get empty frame
3. Swap page into frame
4. Reset tables
5. Set validation bit = **v**
6. Restart the instruction that caused the page fault



Page fault

- Memory access for a legal page not in memory causes a page fault trap.
- Needs to bring the page into memory.

Steps of bringing a page into memory

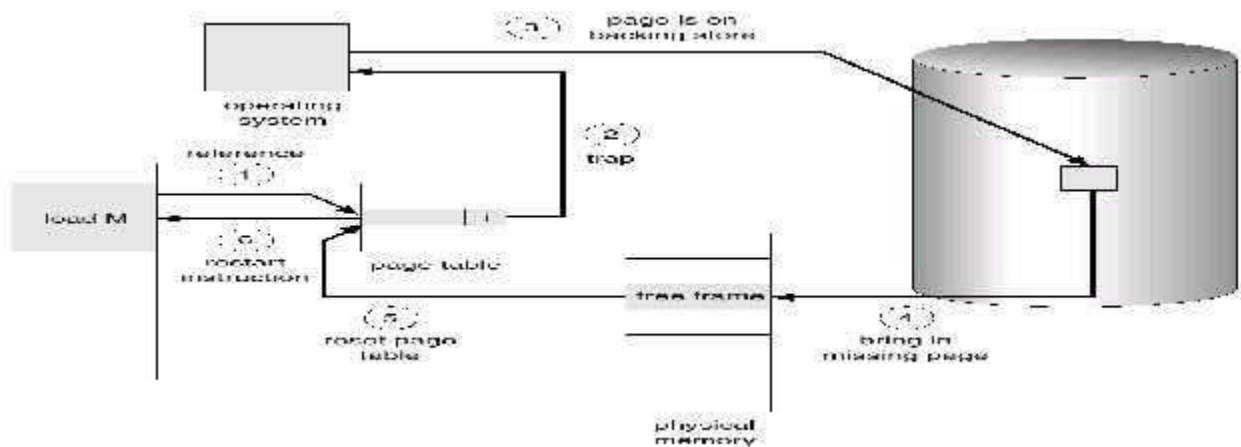
1. Find a free frame
2. Read the desired page from disk into the free frame
3. Update page table valid bit set to v.
4. Restart the interrupted instruction

Pure demand paging

Start executing a process with no pages in memory. Bring a page into memory only when needed.

Steps in handling a page fault:

- We check an internal table for this process to determine whether the reference was valid or invalid memory access.
- If the reference was invalid, we terminate the process. If the reference was valid but we have not yet brought in that page
- Now we find a free frame from free frame list.
- We Schedule a disk operation to read the desired page into newly allocated frame.
- When the disk read is complete, we need to modify the internal table which is kept with the process and page table to indicate the page from the memory.
- We restart the instruction that was interrupted by the illegal address trap.
- The process now can access the page as though it had always been in the memory.
- Restart instruction
- block move
- auto increment/decrement location



In the above figure Page Fault Rate $0 \leq p \leq 1.0$

- if $p = 0$ no page faults
- if $p = 1$, every reference is a fault

Effective Access Time (EAT)

$$EAT = (1-p) \times \text{memory access} + p(\text{page fault overhead} + \text{swap page out} + \text{swap page in} + \text{restart overhead})$$

2. Explain page replacement in detail? (11) (APR'15, NOV '15, NOV '18)

Page replacement

When a fault occurs, the OS loads the faulted page from disk into a page of memory. At some point, the process has used the entire page frames it is allowed to use. When this happens, the OS must *replace* a page for each page faulted in. That is, it must select a page to throw out of primary memory to make room. How it does this is determined by the page replacement algorithm. The goal of the replacement algorithm is to reduce the fault rate by selecting the best victim page to remove.

If total memory requirements exceed the physical memory, then it may be necessary to replace pages from memory to free frames for new pages.

Page replacement algorithms:

1. FIFO Page Replacement
2. Optimal Page Replacement
3. LRU Page Replacement
4. LRU Approximation Page Replacement
 - a. Additional Reference Bits Algorithm
 - b. Second Chance Algorithm:
 - c. Enhanced Second Chance Algorithm
5. Counting Based Page Replacement
6. Page buffering algorithm

To illustrate the page replacement algorithms, we shall use the reference string

7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1

FIFO Page Replacement:

- A FIFO replacement algorithm associates with each page the time when that page was brought into memory.
- When a page must be replaced, the oldest page is chosen.
- We can create a FIFO queue to hold all pages in memory; we replace the page at the head of the queue.
- When the page is brought into the memory it is inserted at the tail of the queue.
- For the given reference string the 3 frames are initially empty.
- The first 3 references (7,0,1) causes page faults and are brought into these empty frames.
- The next reference (2) replaces page 7 because page 7 was brought in first.

The process continuous as below

Given string 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7 7 7 2 2 2 4 4 4 0 0 0 7 7 7

```

0 0 0   3 3 3 2 2 2   1 1   1 0 0
1 1   1 0 0 0 3 3   3 2   2 2 1

```

- The FIFO page replacement algorithm is easy and its performance is not always good.
- This algorithm yields 15 page faults

Optimal Page Replacement:

- An optimal page replacement algorithm has lowest page fault rate of all algorithms.
- An optimal page replacement algorithm exist and is called OPT or MIN.
- It simply replaces the page that will not be used for the longest period of time.
- This algorithm yields 9 page faults.

```

Given string  7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
              7 7 7 2   2   2       2       2       7
              0 0 0   0   4       0       0       0
              1 1   3   3       3       1       1

```

The reference to page 2 replaces page 7 because it will not be used until reference 18 ,whereas page 0 will be used at 5 and page 1 at 14.

- No replacement algorithm can process this string in 3 frames with than 9 faults.
- But this algorithm is difficult to implement it is mainly used in comparison studies.

LRU Page Replacement:

- LRU stands for least recently used
- LRU replacement associates with each page the time of that page's last use.
- When a page must be replaced LRU chooses a page that has not been used for the longest period of time.
- This strategy is apt for looking backward in time.
- The LRU produces 12 page faults .When reference to page4 occurs LRU selects page2 because that was used least recently. Then LRU algorithm replaces page 2 .This process continues for the entire reference string.
- LRU with 12 faults is better than FIFO with 15 faults.
- The major problem with this algorithm is, how to implement LRU replacement .It requires substantial hardware assistance.

- Given string 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1


```

              7 7 7 2   2   4 4 4 0   1 1 1
              0 0 0   0   0 0 3 3   3 0 0

```


1. Counters:

In the simple case, we associated with each page table entry a time of use field and add to the CPU a logical clock or counter. The clock is incremented for every memory reference. We replace the page with the smallest time value. This scheme requires a search of the page table to find for each memory access. The times must also be maintained when page tables are changed.

2. Stack:

Another approach to implementing LRU replacement is to keep a stack of page numbers. Whenever a page is referenced, it is removed from the stack and put on the top. In this way, the most recently used page is always at the top of the stack and the least recently used page is always at the bottom.

Removing a page and putting it on the top of the stack then requires changing six pointers at worst. Each update is a little more expensive, but there is no search for a replacement, the tail pointer points to the bottom of the stack, which is the LRU page.

LRU Approximation Page Replacement:**Additional Reference Bits Algorithm:**

At regular intervals a timer interrupt transfers control to the operating system. The operating system shifts the reference bit for each page into the high order bit of its 8bit byte, shifting the other bits right by 1 bit and discarding the low order bit.

These 8 bit shift registers contain the history of page use for the last eight time periods. If the shift register contains 00000000 for example then the page has not been used for eight time periods a page that is used at least once in each period has a shift register value of 1111111.

The number of bits of history can be varied of course and is selected to make the updating as fast as possible. In the extreme case, the number can be reduced to zero, leaving only the reference bit itself. This algorithm is called the second chance page replacement algorithm.

Second Chance Algorithm:

The basic algorithm of second chance replacement is a FIFO replacement algorithm. When a page has been selected, however, we inspect its reference bit. If the value is 0, we proceed to replace this page, but if the reference bit is set to 1, we give the page a second chance and move on to select the next FIFO page. One way to implement the second chance algorithm is as a circular queue. A pointer indicates which page is to be replaced next.

Enhanced Second Chance Algorithm:

We can enhance the second chance algorithm by considering the reference bit and the modify bit as an ordered pair. With these two bits, we have the following four possible classes:

1. (0,0) neither recently used nor modified then best page to replace.
2. (0, 1) not recently used but modified then not quite as good, because the page will need to be written out before replacement.
3. (1, 0) recently used but clean then probably will be used again soon.
4. (1, 1) recently used and modified then probably will be used again soon, and the page will be need to be written out to disk before it can be replaced.

The major difference between this algorithm and the simpler clock algorithm is that here we give preference to those pages that have been modified to reduce the number of I/Os required.

Counting Based Page Replacement:

A counter of the number of references that have been made to each page and develop the following two schemes.

1. LFU
2. MFU

1. Least Frequently Used (LFU):

The least frequently used (LFU) page replacement algorithm requires that the page with the smallest count be replaced. The reason for this selection is that an actively used page should have a large reference count.

A problem arises, however, when a page is used heavily during the initial phase of a process but then is never used again. Since it was used heavily, it has a large count and remains in memory even though it is no longer needed.

2. Most Frequently Used (MFU):

The most frequently used (MFU) page replacement algorithm is based on the argument that the page with the smallest count was probably just brought in and has yet to be used.

Page Buffering Algorithms:

- a. A system commonly has a pool of free frames.
- b. In this algorithm, when a page fault occurs, a victim page is chosen as before. However the desired page is stored into a free frame before the victim is written out.
- c. This procedure allows the process to restart as soon as possible, without waiting for the victim page to be written out.
- d. When the victim page is later written out then its frame is added to the free frame pool.

3. Explain Thrashing? (APR '11) (APR '14) (NOV '15)

Thrashing occurs when a computer's virtual memory subsystem is in a constant state of paging, rapidly exchanging data in memory for data on disk, to the exclusion of most application level processing. This causes the performance of the computer to degrade or collapse. The situation may continue indefinitely until the underlying cause is addressed.

If a process does not have enough pages, thrashing is a high paging activity, and the page fault rate is high. This leads to low CPU utilization. In modern computers, thrashing may occur in the paging system (if there is not sufficient physical memory or the disk access time is overly long), or in the communications system especially in conflicts over internal bus access. Depending on the configuration and algorithms involved, the **throughput** and **latency** of a system may degrade by multiple orders of magnitude.

Causes

In virtual memory systems, thrashing may be caused by programs or workloads that present insufficient locality of reference: if the working set of a program or a workload cannot be effectively held within physical memory, then constant data **swapping thrashing** may occur. The term was first used during the tape operating system days to describe the sound the tapes made when data was being rapidly written to and read from them. Many older low-end computers have insufficient RAM (memory) for modern usage patterns and increasing the amount of memory can often cause the computer to run noticeably faster. This speed increase is due to the reduced amount of paging necessary.

An example of this sort of situation occurred on the IBM System/370 series mainframe computer, in which a particular instruction could consist of an execute instruction (which crosses a page boundary) that points to a move instruction (which itself also crosses a page boundary), targeting a move of data from a source that crosses a page boundary, to a target of data that also crosses a page boundary. The total number of pages thus being used by this particular instruction is eight, and all eight pages must be present in memory at the same time. If the operating system allocates fewer than eight pages of actual memory, when it attempts to swap out some part of the instruction or data to bring in the remainder, the instruction will again page fault, and it will thrash on every attempt to restart the failing instruction.

Solutions

To resolve thrashing due to excessive paging, a user can do any of the following:

- Increase the amount of RAM in the computer (generally the best long-term solution).
- Decrease the number of programs being run on the computer.
- Replace programs that are memory heavy with equivalents that use less memory.

4. What are files and explain the access methods for files? (APR '14, NOV '15)

- A file is an abstract data type. The operating system can provide system calls to create, write, read, reposition, delete and truncate files.
- A file is a named collection of related information that is recorded on secondary storage.
- A files contains either programs or data.
- File is a sequence of bits, bytes, lines as defined by the files creator and user.
- A file has certain 'structure' based on its type.
- Operating system must do for each of the six basic file operations.

Text files: Sequence of characters organized into lines/pages.

Source file: Sequence of subroutines and functions organized as executable statements.

Object file: Sequence of bytes organized into blocks understandable by systems linker.

Executable file: Series of code sections that loader can bring into memory and execute.

File attributes:

Name : only information kept in human – readable

Identifier : unique tag identifies file within file system.

Type : needed for systems that support different types.

Location : pointer to file location on device.

Size : current file size.

Protection : controls who can do reading, writing, executing.

File operations:

Time, date and user identification: data for protection, security and usage monitoring.

1. Creating a file
2. Writing a file
3. Reading a file
4. Repositioning within a file
5. Deleting a file
6. Truncating a file

Creating a file:

- Two steps are necessary to create a file
- space in the file system must be found for the file
- an entry for the new file must be made in the directory
- The directory entry records the name of the file and the location in the file system, and possibly other information.

Writing a file:

- To write a file, we make a system call specifying both the name of the file and the information to be written to the file.
- The system searches the directory to find the location of the file.
- The system must keep a write pointer to the location in the file where the next write is to take place.
- The write pointer must be updated whenever a write occurs.

Reading a file:

- To read from a file, we use a system call that specifies the name of the file and where(in memory) the next block of the file should be put.
- The directory is searched for the associated directory entry, and the system needs to keep a read pointer to the location in the file where the next read is to take place.
- Once the read has taken place, the read pointer is updated.
- The current operation location is kept as a per-process current-file-position-pointer.

Repositioning within a file:

- The directory is searched for the appropriate entry, and the current-file-position is set to a given value.
- Repositioning within a file does not need to involve any actual I/O.
- This file operation is also known as file seek.

Deleting a file:

To delete a file, we search the directory for the named file. Having found the associated directory entry, we release all file space, so that it can be reused by other files, and erase the directory entry.

Truncating a file:

The user may want to erase the contents of a file but keep its attributes. Rather than forcing the user to delete the files and the recreate it, this function allows all attributes to remain unchanged.

- These six basic operations certainly comprise the minimal set of required file operations. Other common operations include appending new information to the end of an existing file and renaming an existing file. Most of the file operations mentioned involve searching the directory for the entry associated with the named file.
- The operating system keeps a small call be used before that file is first about all open files. Pieces of information are associated with an open file.

File pointer:

This pointer is unique to each process operating on the file and therefore must be kept separate from the on disk file attributes.

File open count:

This counter tracks the number of opens and closes and reaches zero on the last close.

The system can then remove the entry.

Disk location of the file:

The information needed to locate the file on disk is kept in memory to avoid having to read it from disk for each operation.

Access rights:

The information is stored on the per-process table so the operating system can allow or deny subsequent I/O requests.

Access Methods:

- It is used to access the information of file and read into computer memory.
- The information of file can be accessed in many ways.
- Some systems provide only one access methods for file.

There are 2 access methods

1. Sequential access
2. Direct access

Sequential access:

- Sequential access is simplest access method.
- Information in the file is processed in order, by one after the other.
- Read and write made up of the bulk of operations on a file
- Read operation: reads the next portion of the file and automatically advances a file pointer, which tracks the I/O location.
- Write operation: appends to the end of the file and advances to the end of newly written material.
- File can be reset to beginning and on some system ,a program may be skip forward or backward n records for integer
- Sequential access file is shown in figure Beginning current position end
- Sequential access file is based on tape model of file.

Direct access:

- Direct access is also called as relative access.

- A file is made up of fixed length logical records that allow programs to read and write records rapidly.
- It is based on disk method of a file, since disk allow random access to any file block
- In direct access, file is viewed as numbered sequence of blocks or records
- In direct access, there is no restriction for reading and writing.
- It is great use for immediate access to large amounts of information.
- In this method, the file operation must be modified to include the block number as a parameter.
- Thus, we have read n where block number is rather than read next.
- This block number provide by the user to operating system is normally a relative block number.
- Relative block number is a index relative to the beginning of the file.
- Not all operating system support both sequential and direct access for files.
- Some system require that a file as to be defined as sequential or direct access when it is created so such file can be accessed by consistent with its declaration.

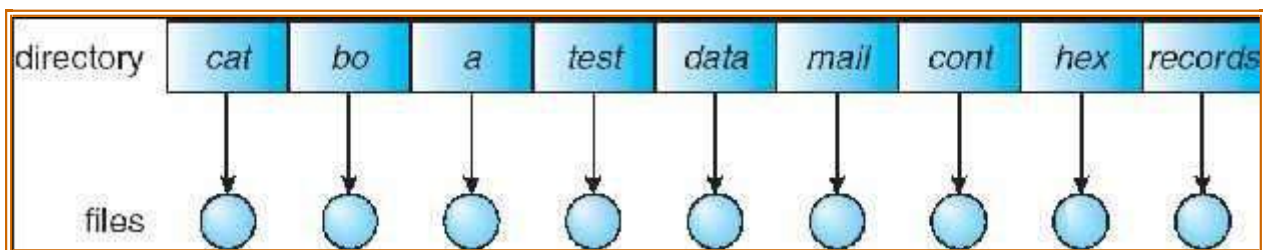
Other access methods:

It can be built on top of direct access method and involve the construction of an index for the file. The index contains pointers to various blocks. To find the desired record in the file, we first search the index and then use the pointers to access the file directly.

5. Write short notes on single and two-level Directory? (APR '11)

Single level directory:

The Single level directory is simplest directory structure because is easy to support and understand files in system. It as limitation when the number of files increase or when the system as more than one user .All files are in same directory so they must have unique names. Even a single user on Single -level directory may find difficult to remember names of all files as number of files increase. It keep tracks the files is a daunting tasks.



Two level directory:

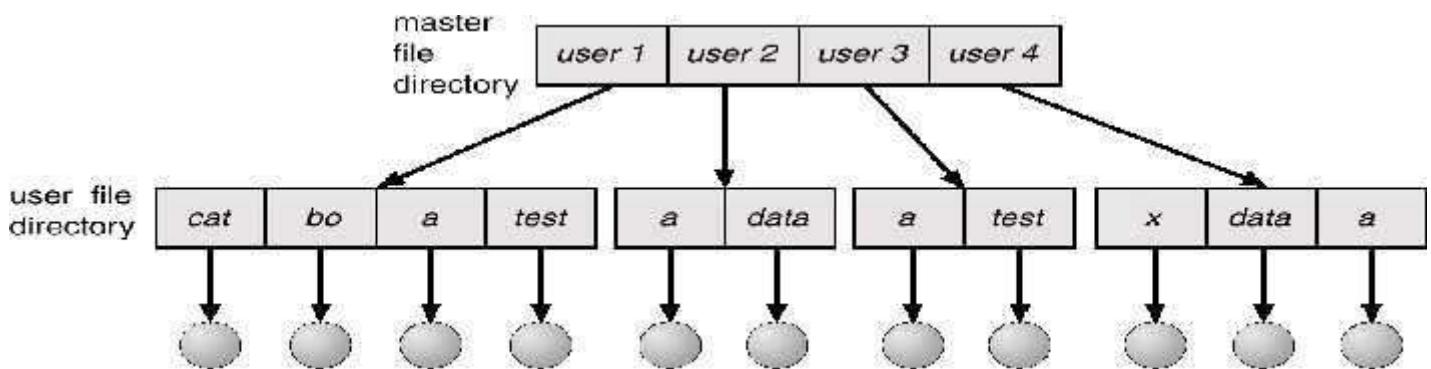
- In the two level directory structure, each user has his own user file directory(UFD).
- The UFDs have similar structures, but each lists only he files of single user.
- User job starts or a user logs in, the system's mater file directory (MFD) is searched.

- Indexed by user name or account number.
- All the file names within each UFD are unique.
- To delete a file, the operating system confines its search to the local UFD. The program creates a new UFD and adds an entry for it to then MFD.

The two level directory structures solves the name collision problem, it still has disadvantages.

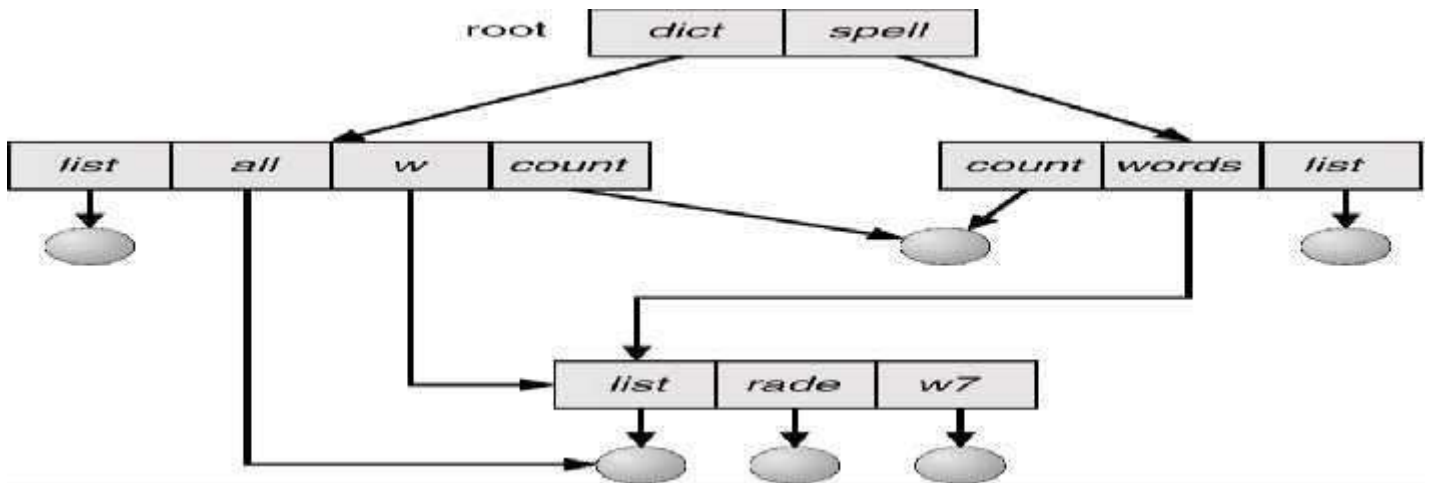
Effectively isolates one user form another.

- Isolation is an advantage when the users are completely independent.
- A disadvantage when the users want to cooperate on some task and to access one another's files.
- Some systems simply do not allow local user files to be accessed by other users.
- This method is the one most used in UNIX and MS-DOS.



6. Describe about Acyclic Graph Directory (11 Marks) (NOV 13)

- A tree structure prohibits the sharing of files or directories.
- An acyclic graph that is, a graph with no cycles allows directories to share subdirectories and files. The same file or subdirectory may be in tow different directories.
- The acyclic graph is a natural generalization of the tree structured directory scheme.
- It is important to note that a shared file is not the same as two copies of the file.
- With two copies, each programmer can view the copy rather than the original, but if one programmer changes the file, the changes will not appears in the other's copy.
- With a shared file, only one actually file exists, so any changes made by one persons are immediately visible to the other.
- Sharing is particularly important for subdirectories; a new file created by one person will automatically appear in all the shared subdirectories.
- A link is effectively a pointer to another file or subdirectory.

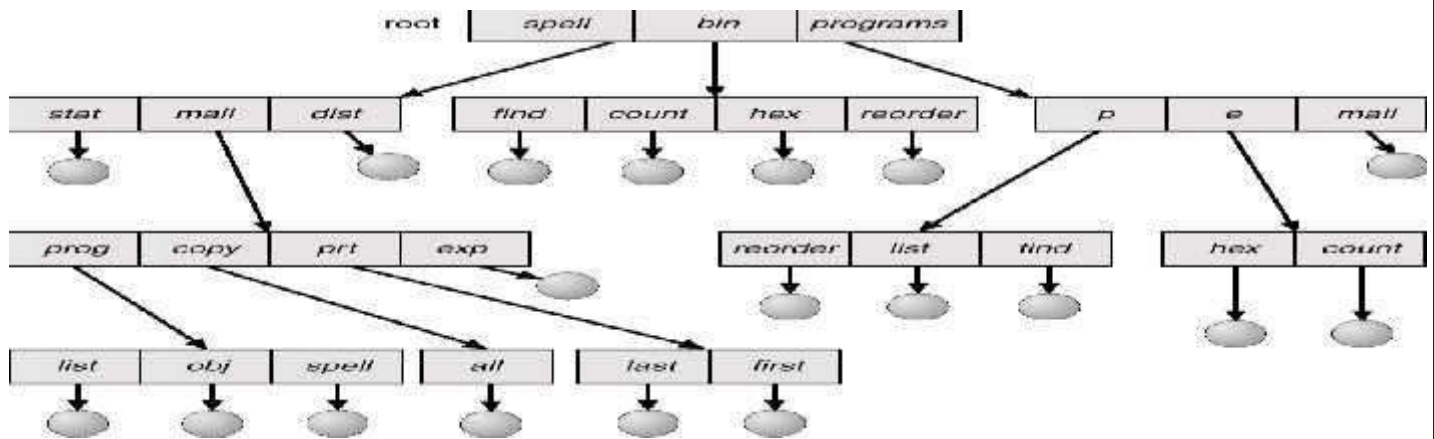


- We resolve the link by using that path name to located the real file.
- Links are easily identified by their format in the directory entry and are effectively named indirect pointers.
- The operating system structure of the system.
- An acyclic graph directory structures is more flexible than is a simple tree structure, but it is also more complex.
- Several problems must be considered carefully. A file may now have multiple absolute path names. Consequently, distinct file names may refer to the same file.
- This situation is similar to the **aliasing problem** for programming languages.

7. Explain Tree structured directories? 5 Marks (APR '14)

- This generalization allows users to create their own subdirectories and to organize their files accordingly.
- To extend the directory structure to a tree of arbitrary height.
- The tree has a root directory, and every file in the system has a unique path name. a directory contains a set of files or subdirectories.
- All directories have the same internal format.
- The current directory should contain most of the files that are of current interest to the process.
- If a file is needed that is not in the current directory, then the user usually must either specify a path name or change the current directory to be the directory holding that file.
- To change directories, a system call is provided that takes a directory name as parameter and uses it to redefine the current directory.
- Path names can be of two types: absolute and relative.
- An absolute path name begins at the root and follows a path down to the specified file, giving the directory names on the path.

- A relative path name defines a path from the current directory.
- A user to define her own subdirectories permits her to impose a structure on her files.
- This structure might result in separate directories for files associated with different topics or different forms of information.



8. Write notes about the protection strategies provided for files . (APR'15)

When information is stored in a computer system, we want to keep it safe from physical damage (the issue of *reliability*) and improper access (the issue of *protection*). Reliability is generally provided by duplicate copies of files. Many computers have systems programs that automatically (or through computer-operator intervention) copy disk files to tape at regular intervals (once per day or week or month) to maintain a copy should a file system be accidentally destroyed. File systems can be damaged by hardware problems (such as errors in reading or writing), power surges or failures, head crashes, dirt, temperature extremes, and vandalism. Files may be deleted accidentally. Bugs in the file-system software can also cause file contents to be lost. Reliability .Protection can be provided in many ways. For a small single-user system, protection provided by physically removing the floppy disks and locking them in a desk drawer or file cabinet. In a multiuser system, however, other mechanisms are needed.

Types of Access

The need to protect files is a direct result of the ability to access files. Systems that do not permit access to the files of other users do not need protection. Thus, we could provide complete protection by prohibiting access. Alternatively, we could provide free access with no protection. Both approaches are too extreme for general use. What is needed is Protection mechanisms provide controlled access by limiting the types of file access that can be made. Access is permitted or denied depending on several factors, one of which is the type of access requested. Several different types of operations may be controlled:

Read. Read from the file.

Write. Write or rewrite the file.

Execute. Load the file into memory and execute it.

Append. Write new information at the end of the file.

Delete. Delete the file and free its space for possible reuse.

List. List the name and attributes of the file.

Other operations, such as renaming, copying, and editing the file, may also be controlled. For many systems, however, these higher-level functions may be implemented by a system program that makes lower-level system calls. Protection is provided at only the lower level. For instance, copying a file may be implemented simply by a sequence of read requests. In this case, a user with read access can also cause the file to be copied, printed, and so on.

Many protection mechanisms have been proposed. Each has advantages and disadvantages and must be appropriate for its intended application. A small computer system that is used by only a few members of a research group, for example, may not need the same types of protection as a large corporate computer that is used for research, finance, and personnel operations.

Access control

The most common approach to the protection problem is to make access dependent on the identity of the user. Different users may need different types of access to a file or directory. The most general scheme to implement dependent access is to associate with each file and directory specifying user names and the types of access allowed for each user. When a user requests access to a particular file, the operating system checks the access list associated with that file. If that user is listed for the requested access, the access is allowed. Otherwise, a protection violation occurs, and the user job is denied access to the file.

The access control approach has the advantage of enabling complex access methodologies. The main problem with access lists is their length. If we want to allow everyone to read a file, we must list all users with read access. This technique has two undesirable consequences:

1. Constructing such a list may be a tedious and unrewarding task, especially if we do not know in advance the list of users in the system.
2. The directory entry, previously of fixed size, now must be of variable size, resulting in more complicated space management.

These problems can be resolved by use of a condensed version of the access list. To condense the length of the access-control list, many systems recognize three classifications of users in connection with each file:

Owner. The user who created the file is the owner.

Group. A set of users who are sharing the file and need similar access is a group, or work group.

Universe. All other users in the system constitute the universe.

The most common recent approach is to combine access-control lists with the more general (and easier to implement) owner, group, and universe access control scheme . For example, Solaris 2.6 and beyond use the three categories of access by default but allow access-control lists to be added to specific files and directories when more fine-grained access control is desired.

Other Protection Approaches

Another approach to the protection problem is to associate a password with each file. Just as access to the computer system is often controlled by a password, access to each file can be controlled in the same way. If the passwords are chosen randomly and changed often, this scheme may be effective in limiting access to a file.

9. Consider the following page reference string: (APR '11)

1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6.

How many page faults would occur for the following replacement algorithms, assuming one, two, three, four, five, six, or seven frames?

Remember all frames are initially empty, so your first unique pages will all cost one fault each.

LRU replacement

FIFO replacement

Optimal replacement

Answer:

Number of frames	LRU	FIFO	Optimal
1	20	20	20
2	18	18	15
3	15	16	11
4	10	14	08
5	08	10	07
6	07	10	07
7	07	07	07

10. Explain in detail about file sharing? (APR '12)

Sharing of files on multi-user systems is desirable. Sharing may be done through a protection scheme. On distributed systems, files may be shared across an Network. Network File System (NFS) is a

common distributed file sharing method. User IDs identify users, allowing permissions and protections to be per user. Group IDs allow users to be in groups, permitting group access rights.

File Sharing –Remote File Systems

Uses networking to allow file system access between systems in manually via programs like FTP Automatically, seamlessly using distributed file systems Semi automatically via the World Wide Web. Client server: model allows clients to mount remote file systems from servers Server can serve multiple clients Client and user on client identification is insecure or complicated NFS is standard UNIX client server file sharing protocol .CIFS is standard Windows protocol Standard operating system file calls are translated into remote calls Distributed Information Systems (distributed naming services) such as LDAP, DNS, NIS implement unified access to information needed for remote computing.

File Sharing –Failure Modes

Remote file systems add new failure modes, due to network failure, server failure. Recovery from failure can involve state information about status of each remote request. Stateless protocols such as NFS include all information in each request, allowing easy recovery but less security.

File Sharing –Consistency Semantics

Consistency semantics specify how multiple users are to access a shared file simultaneously process synchronization algorithms tend to be less complex due to disk I/O and network latency (for remote file systems Andrew File System (AFS) implemented complex remote file sharing semantics Unix file system (UFS) implements: Writes to an open file visible immediately to other users of the same open file ,Sharing file pointer to allow multiple users to read and write concurrently AFS has session semantics Writes only visible to sessions starting after the file is closed.

Semantics of File Sharing

UNIX semantics: used in centralized systems. UNIX semantics: a read that follows two writes in quick Succession sees the result of the last write. Semantics of File Sharing is issues in Distributed File Systems. In a Single File Server there is no client caching easy to implement UNIX semantics Client File Caching improves performance by decreasing demand at the server updates to the cached file are not seen by other clients.

Session Semantics: (relaxed semantics) changes to an open file are only visible to the process that modified the file. When the file is closed, changes are visible to other processes closed file is sent back to the server. Two or more clients are caching and modifying a file a final result depends on who closes last use an arbitrary rule to decide who wins. In this file pointer sharing not possible when a process and its children run on different machines.

No file update semantics :

No File Updates Semantics: files are never updated. Allowed file operations: CREATE and READ files are atomically replaced in the directory.

Transaction Semantics: all file changes are delimited by a Begin and End transaction. All file requests within the transaction are carried out in order. The complete transaction is either carried out completely or not at all (atomicity).

UNIX Semantics every operation is instantly visible to others. Session Semantics no changes visible until file is closed. In No Updates Semantics no file updates are allowed. In Transactions atomic updates for file.

Summary:

UNIX SEMANTICS :every operation is instantly visible to others.

SESSION SEMANTICS :no changes visible until file is closed.

NO FILE UPDATE SEMANTICS :no file updates are allowed.

TRANSACTION SEMANTICS :atomic updates for file is allowed.

11. Write short notes on process creation**Process creation:**

Virtual memory provides two techniques to enhance performance of creating and running processes.

- ➔ Copy –on –write
- ➔ Memory mapped files.

Copy on write:

- Demand paging is used when reading a file from disk to memory.
- Process creation using fork() system call may by pass the need for demand paging.
- Alternatively a technique copy – on – write can be used.
- This works by allowing parent and child processes to initially share the same pages.
- These shared pages are marked as copy on writ pages if either process writes to a shared page, copy of the shared page is created.
- Using this technique only the pages that are modified by either process are copied.
- Non modified pages may be shared by parent and child processors.
- It is a common technique used by windows 2000, Linux, Solaris 2.

Memory mapped files:

- Sequential read of a file on disk requires standard system calls open(), read(), write().
- Alternatively virtual memory technique memory mapping can be used.
- This allows a part of the virtual address space to be logically associated with a file.
- Initial access to the file proceeds using ordinary demand paging, resulting in page fault.

- Subsequent reads and writes to the file are handled as routine memory accesses.
- Closing the file result in all the memory mapped data being written back to disk and removed from virtual memory of the process.

Eg: Soloris 2 operating system uses this technique.

12. Explain how many frames can be allocated to processors?

ALLOCATION OF FRAMES:

- The operating system allocates all its buffer and table space from the free – frame list.
- When this space is not in use by the operating system, it can be used to support user paging.
- Three free frames can be reserved on the free- frame list at all the times.
- When a page fault occurs there will be a free frame available.
- Different problem arises when demand paging is combined with multi – programming.
- As multiprogramming puts 2(or more) processes at same time.

Minimum number of frames:

- **As the** number of frames allocated to each process decreased the page fault rate increases, slowing processes execution.
- At least a minimum number is defined by the instruction set architecture.
- The maximum number is defined by the amount of available physical memory.

Allocation algorithms:

- **Equal allocation:** split 'm' frames among n processes
- **Equal share**-> m/n frames.
- Left over frames free frame buffer
- **Proportional allocation:**allocate available memory to each processes according to its size.
- $a_i = s_i / s * m$
- a_i frames
- s_i size of memory.
- M available frames

Global versus local allocation :

Global replacement:

- Allows process to select a replacement frames from the set of all frames even if that frame is currently allocated to some other process.
- One process can take a frame from another.
- One problem is that a process cannot control its own page fault rate.
- Results in greater system throughput ; a most common method.

Local replacement:

- Requires that each process select from only its own set of allocated frames.
- The number of frames allocated to a process does not change.
- The set of pages in memory for a process is affected by the paging behavior of only that process.

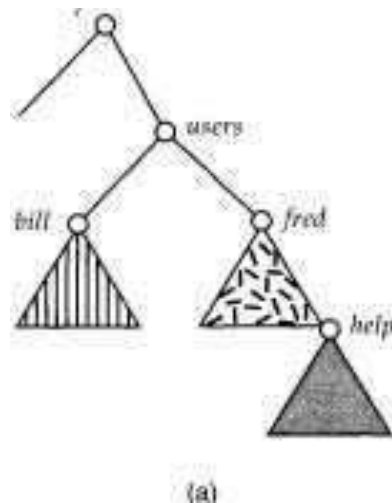
13. Explain about file system mounting

FILE SYSTEM MOUNTING:

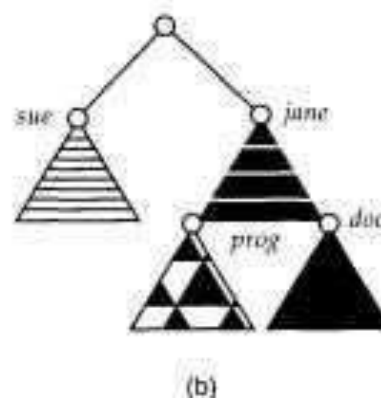
- Similar to a file open process before using it a file system must be mounted before it can be available to processes on the system.
- The directory structure can be built out of multiple partitions which must be mounted to make them available within the file system name space.

PROCEDURE:

- **Os** is given the name of the device; location within the files structure to which to attach the file system(mount point).
- A mount point is an empty directory



File system existing



Un mounted partition

MOUNT POINT:

- A system may allow the same file system to be mounted repeatedly ; at different mount points or it may only allow one mount per file system.

Pondicherry University Questions

2 Marks

1. What is virtual memory?(APR '11)(APR '12) (Ref.Pg.No.1 Qn.No.1)
2. Compare Swapper with Lazy Swapper?(NOV '13) (Ref.Pg.No.1 Qn.No.4)
3. What is thrashing? (APR 11) (NOV '15) (Ref.Pg.No.3 Qn.No.12)
- 4 Enumerate Search Path?(UQ NOV '13) (Ref.Pg.No.3 Qn.No.15)
5. List the various File Attributes? (UQ APR '12 &NOV '12) (Ref.Pg.No.4 Qn.No.19)
6. What is System file checker in Windows?(NOV '11) (Ref.Pg.No.7 Qn.No.31)
7. Give any two criteria to chose a file organization? (APR '12) (Ref.Pg.No.7 Qn.No.32)
8. What is File Authentication?(NOV '11)(APR '11) (Ref.Pg.No.10 Qn.No.44)
9. What is File Lock?(NOV '11) (APR '11) (Ref.Pg.No.9 Qn.No.77)
- 10.What is File Control Block? (APR '14) (Ref.Pg.No.9 Qn.No.38)
11. What are the advantages and disadvantages of access control? (NOV'14) (Ref.Pg.No.6 Qn.No.25)
12. State the functions of working set strategy model (NOV'14) (Ref.Pg.No.3 Qn.No.14)
- 13.What are the different accessing methods of a file? (APR'15) (Ref.Pg.No.6 Qn.No.23)
14. What is the principle of optimality? (NOV '15) (Ref.Pg.No.11 Qn.No.47)
15. What is demand paging? (NOV '18) (Ref.Pg.No.1 Qn.No.2)
16. what is mounting? (NOV '18) (Ref.Pg.No.10 Qn.No.43)

11 MARKS

- 1.What are files and explain the access methods for files? (APR '14) (NOV '15) (Ref.Pg.No.20 Qn.No.4)
2. Write short notes on single and two-level Directory? (APR '11) (Ref.Pg.No.23 Qn.No.5)
3. Describe about Acyclic Graph Directory (NOV 13) (Ref.Pg.No.24 Qn.No.6)
4. Explain Tree structured directories? (APR '14) (Ref.Pg.No.25 Qn.No.7)
5. Explain Thrashing? (APR '11) (APR '14) (NOV '15) (Ref.Pg.No.19 Qn.No.3)
6. Explain in detail about file sharing (APR '12) (Ref.Pg.No.28 Qn.No.10)
7. Consider the following page reference string
1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6.
How many page faults would occur for the following replacement algorithms, assuming one, two, three, four, five, six, or seven frames? : (APR '11) (Ref.Pg.No.28 Qn.No.9)
8. Explain page replacement in detail? (APR'15) (NOV '15, NOV '18) (Ref.Pg.No.15 Qn.No.2)
9. Write notes about the protection strategies provided for files. (APR'15) (Ref.Pg.No.26 Qn.No.8)

